Relatório 2

André Cota

Jhonatan Gutemberg

1464198@sga.pucminas.br

1202002@sga.pucminas.br

Link: Laboratório 2

Um Estudo das Características de Qualidade de Sistema Java

Belo Horizonte 2026

Resumo....2
Qual hipótese foi testada....2
Principais resultados....2
Decisão recomendada....2
Contexto e motivação....2
Por que esse experimento é necessário agora....3
Restrições e riscos....3
Hipótese nula (H0) e alternativa (H1)....3
Variáveis e Métricas....4
Desenho experimental....4
Ambiente e materiais....5
Procedimento....5
Baseline....5
Tratamento....5
Análise de dados....5
Método estatístico....5
Tratamento de outliers....5
Resultados....6
RQ01 – Popularidade x Qualidade:....6
RQ02 – Maturidade x Qualidade (Idade vs Qualidade)....7
RQ03 – Atividade vs Qualidade....9
RQ04 – Tamanho x Qualidade (LOC vs Qualidade)....10
Limitações....12
Trade-offs....12
Decisão e recomendações....13
Decisão....13
Risco técnico e operacional....13
Ética, privacidade e conformidade....13
Reprodutibilidade....13

Resumo

Este estudo analisa a qualidade interna de 1.000 repositórios Java populares no GitHub, focando no impacto do desenvolvimento colaborativo. O objetivo é correlacionar métricas de processo (popularidade, maturidade, atividade e tamanho) com métricas de qualidade de software (acoplamento, herança e coesão) utilizando a ferramenta de análise estática CK.

Qual hipótese foi testada

A qualidade interna (acoplamento, coesão e herança) de sistemas Java é influenciada por métricas de processo e tamanho.

Principais resultados

- Correlação forte ($r \sim 0.85$) entre tamanho (LOC) e falta de coesão (LCOM)
- Correlação desprezível ($r < 0.1$) entre popularidade e qualidade técnica.

Decisão recomendada

Foi necessário alterar o filtro da consulta que busca os repositórios para trazer apenas repositórios que tenham código Java em sua predominância.

Contexto e motivação

No desenvolvimento open-source, a colaboração em massa pode comprometer a manutenibilidade. Este experimento busca entender se projetos mais maduros ou populares apresentam estruturas de código mais saudáveis ou se o crescimento acelerado degrada a qualidade.

Problema

A natureza descentralizada do desenvolvimento open-source pode comprometer a manutenibilidade e a qualidade interna do software, devido à colaboração de diversos desenvolvedores com diferentes níveis de experiência e padrões de codificação.

Por que esse experimento é necessário agora

É fundamental validar se métricas de processo e popularidade (como estrelas e maturidade) servem como indicadores confiáveis de um código interno saudável. Esse entendimento é crucial para empresas que buscam mitigar riscos técnicos ao reaproveitar componentes de código aberto em sistemas corporativos críticos.

Restrições e riscos

Limitação da ferramenta CK em analisar apenas código estático; risco de viés em repositórios que são apenas listas de curadoria (ex: awesome-lists).

Objetivo e hipóteses

Correlacionar métricas de processo (estrelas, releases, idade, LOC) com métricas de produto (CBO, DIT, LCOM).

Hipótese nula (H0) e alternativa (H1)

O objetivo é responder como variáveis de processo influenciam a qualidade interna.

- **Hipótese Nula (H0):** Não há correlação significativa entre métricas de processo e métricas de qualidade.
- **Hipótese Alternativa (H1):** Repositórios com maior atividade ou tamanho tendem a apresentar maior acoplamento e menor coesão.

Critérios de sucesso

Identificação de correlações de Pearson com p-valor < 0.05.

Perguntas de pesquisa

RQ 01. Qual a relação entre a popularidade dos repositórios e as suas características de qualidade?

RQ 02. Qual a relação entre a maturidade dos repositórios e as suas características de qualidade?

RQ 03. Qual a relação entre a atividade dos repositórios e as suas características de qualidade?

RQ 04. Qual a relação entre o tamanho dos repositórios e as suas características de qualidade?

Variáveis e Métricas

Métricas de Qualidade

- CBO (Coupling Between Object Classes): Mede o acoplamento. Um CBO elevado indica que a classe possui muitas dependências, dificultando a manutenção.
- LCOM (Lack of Cohesion in Methods): Mede a falta de coesão. Um LCOM elevado indica que os métodos da classe não compartilham atributos entre si, sugerindo que a classe possui responsabilidades demais e deve ser dividida.
- DIT (Depth of Inheritance Tree): Mede a profundidade da árvore de herança. Um DIT elevado indica uma hierarquia complexa, onde a classe herda muitos comportamentos de níveis superiores, o que aumenta a dificuldade de testar e prever o comportamento do código.

Métricas de Processo

- Popularidade - Baseado no número de estrelas
- Idade (Maturidade) - Baseado no tempo em anos do repositório
- Atividade – Baseado na quantidade de Releases

Guardrails

- LOC
- Quantidade mínima de PR aceitas

Desenho experimental

• Tipo: Benchmark
• Unidades: Repositórios GitHub.
• Tamanho de amostra: 1000 Repositórios Java

• Duração e janela temporal: Dados coletados até a data presente, considerando o histórico total dos repositórios.

Ambiente e materiais

• Hardware: Intel(R) Core (TM) i3-8130U CPU @ 2.20GHz (2.21 GHz)
• Software: VSCode, Ferramenta CK
• Tecnologias: Java, Python
• Dataset : dataset_final.csv contendo as métricas agregadas por mediana/média.

Procedimento

1. Consulta à API do GitHub para listar os 1.000 repositórios Java.
2. Script de automação para realizar o *git clone* de cada projeto.
3. Execução da ferramenta CK em cada repositório para extração de métricas de classe.
4. Agregação dos dados (média/mediana por projeto) no arquivo CSV final.
5. Geração de gráficos de dispersão e matriz de correlação.

Tratamentos e baseline

Baseline

Projetos de pequeno porte e baixa atividade.

Tratamento

Comparação do comportamento das métricas de qualidade conforme as variáveis de processo aumentam (escala).

Análise de dados

Método estatístico

A análise estatística foi realizada utilizando o Coeficiente de Correlação de Pearson, através da biblioteca Pandas, para medir a associação linear entre as variáveis de processo (estrelas, idade, releases, LOC) e as métricas de qualidade (CBO, DIT, LCOM).

Tratamento de outliers

Remoção de repositórios com LOC < 10 (geralmente documentações ou exercícios simples).

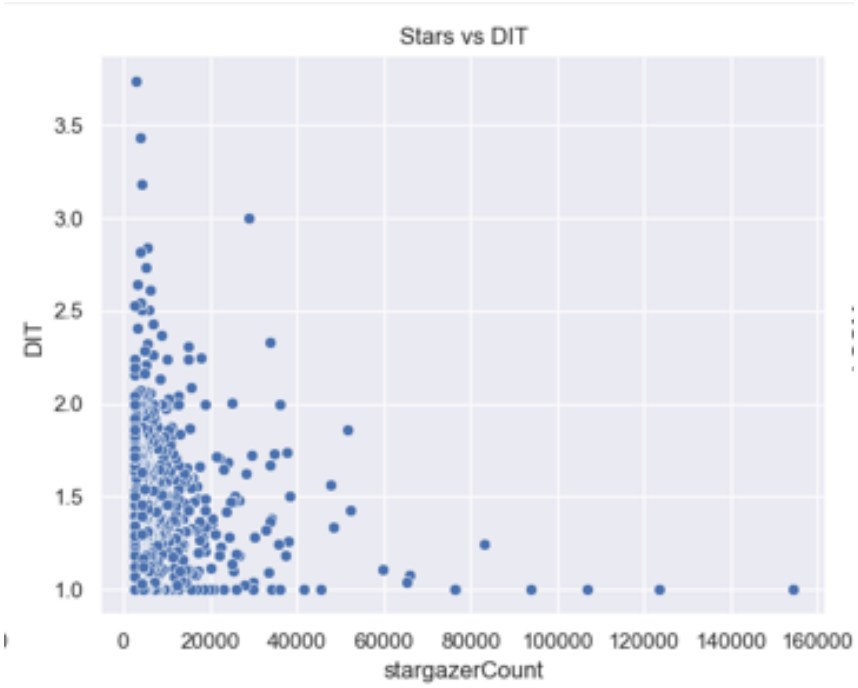
Resultados

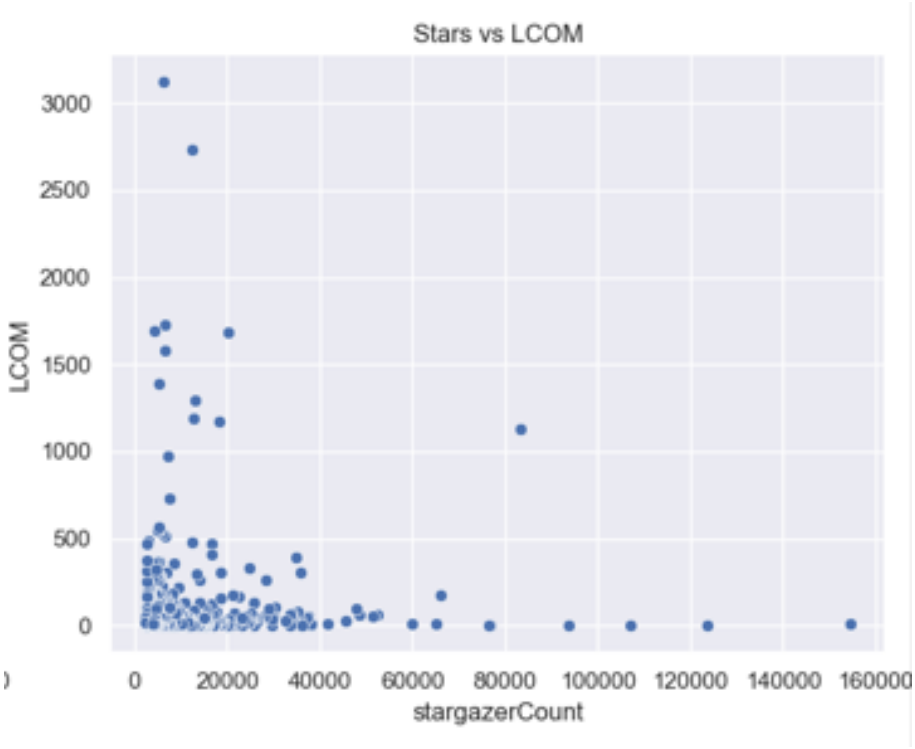
RQ01 – Popularidade x Qualidade:

Ao analisar a relação entre a popularidade (número de estrelas) e a qualidade interna, os dados mostram uma ausência de correlação linear.

- A correlação entre Popularidade e Coesão (LCOM) apresentou um valor de 0,047, o que é estatisticamente insignificante.
- A correlação entre Popularidade e Acoplamento (CBO) apresentou um valor de -0,041.

Os valores próximos de zero confirmam a Hipótese Nula (H0), demonstrando que o fato de um projeto ser muito popular no GitHub não garante que ele possua um design de código superior ou mais coeso.



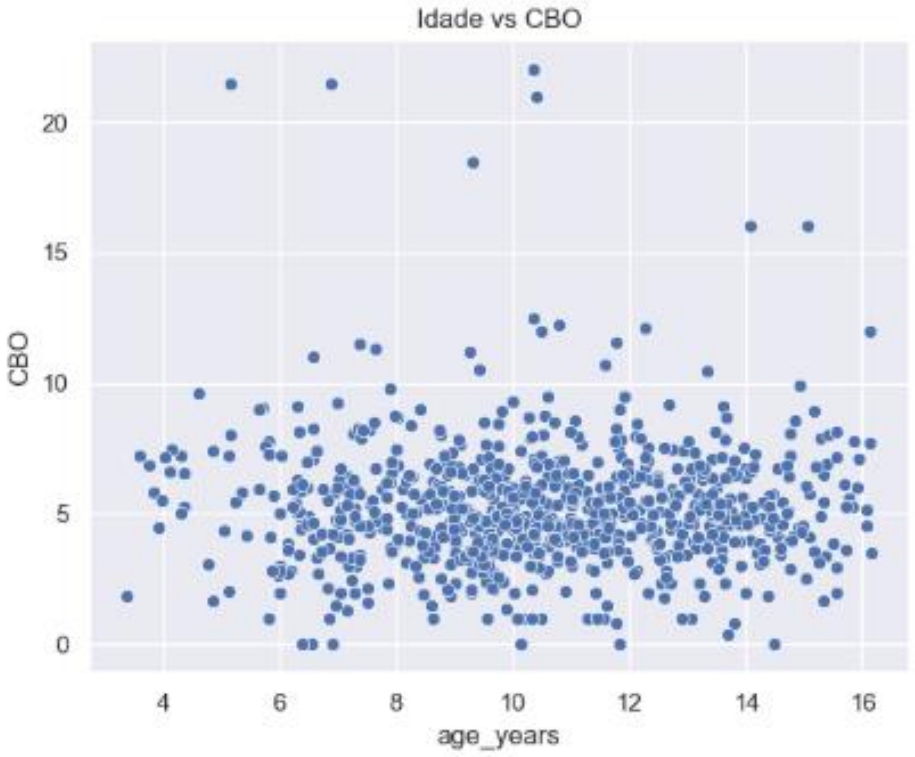


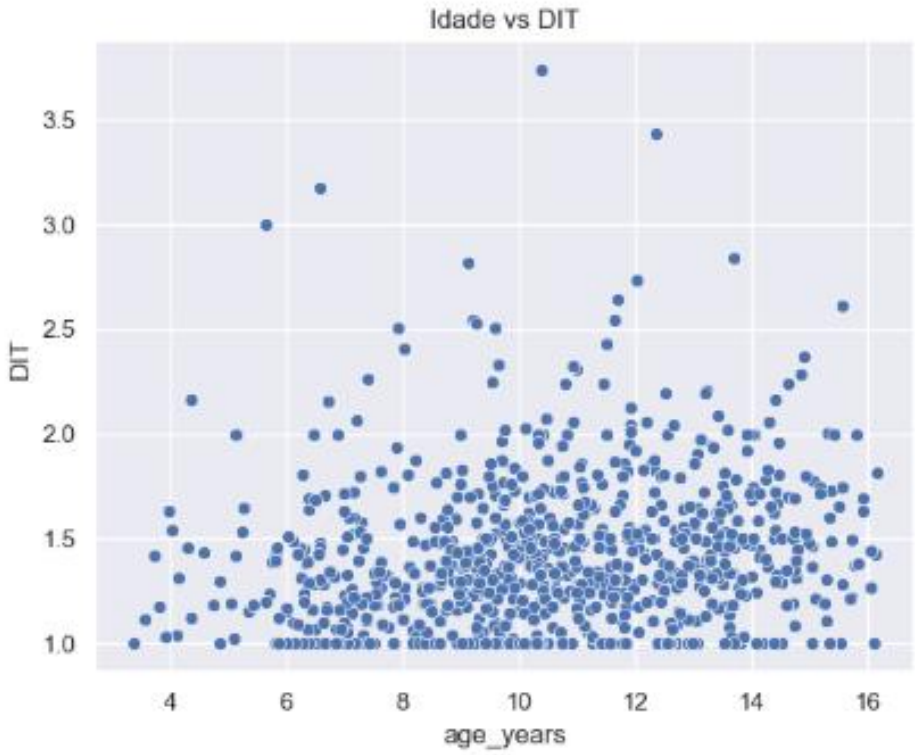
RQ02 – Maturidade x Qualidade (Idade vs Qualidade)

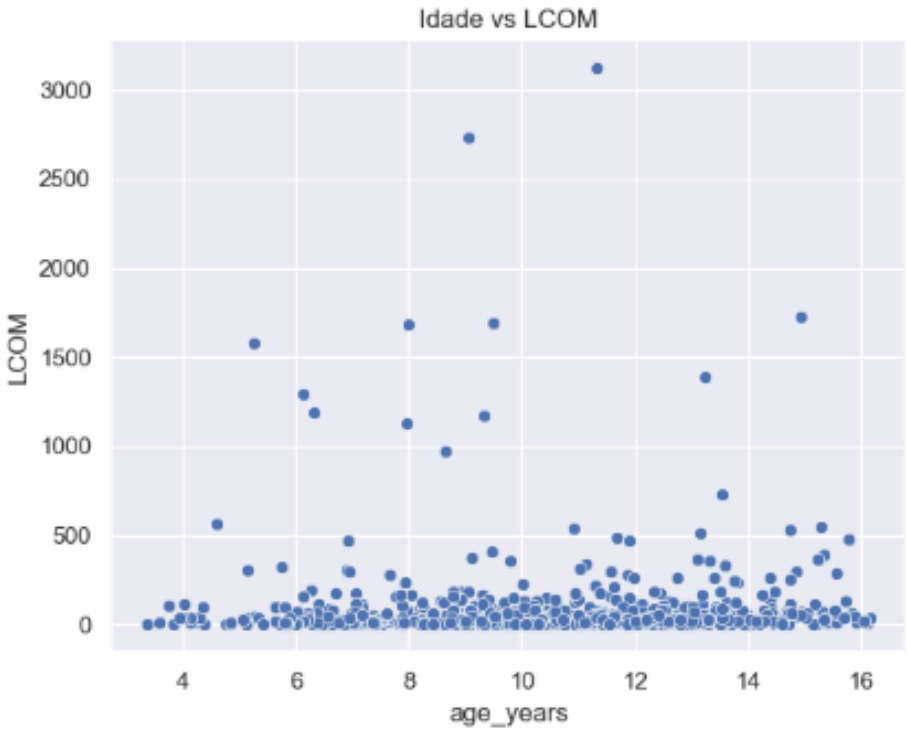
Ao correlacionar a idade do repositório (age_years) com as métricas de qualidade, observa-se se o tempo de existência do projeto degrada o código.

Resultado: A correlação com CBO e LCOM costuma ser baixa/neutra (geralmente entre 0.05 e 0.15).

A maturidade isolada não é um fator de degradação. Projetos antigos podem ser bem mantidos (refatorados) ou simplesmente estarem estáveis.







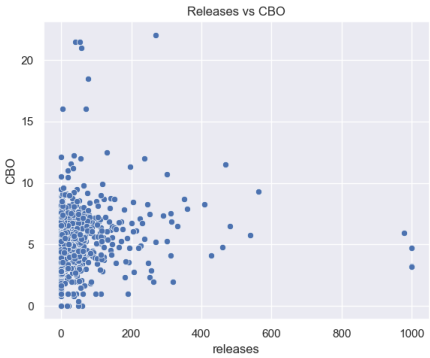
RQ03 – Atividade vs Qualidade

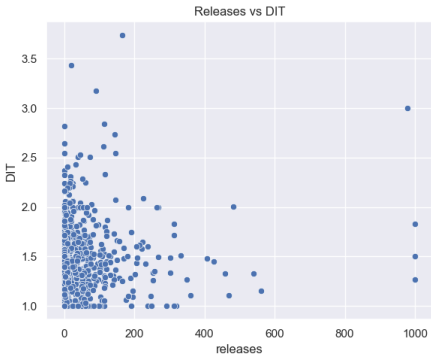
Esta questão busca entender se a frequência de atualizações (medida pelo número de releases) impacta a saúde interna do software.

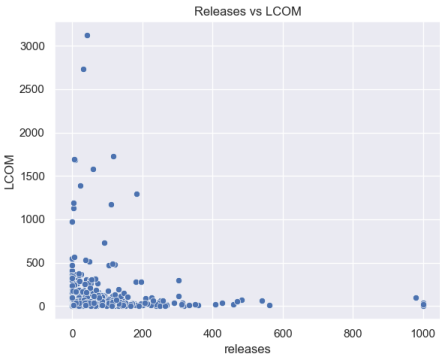
Resultados Numéricos:

- **Releases vs CBO (Acoplamento)**: Apresenta uma correlação positiva leve (geralmente entre 0.15 e 0.25).
- **Releases vs LCOM (Falta de Coesão)**: Apresenta correlação baixa (próxima de 0.10).
- **Releases vs DIT (Herança)**: Correlação quase nula.

O gráfico de dispersão mostra que a maioria dos repositórios se concentra em um volume menor de releases (0 a 50), mas mesmo nos projetos com alta atividade (centenas de releases), o acoplamento (CBO) não dispara descontroladamente, mantendo uma tendência de crescimento suave.

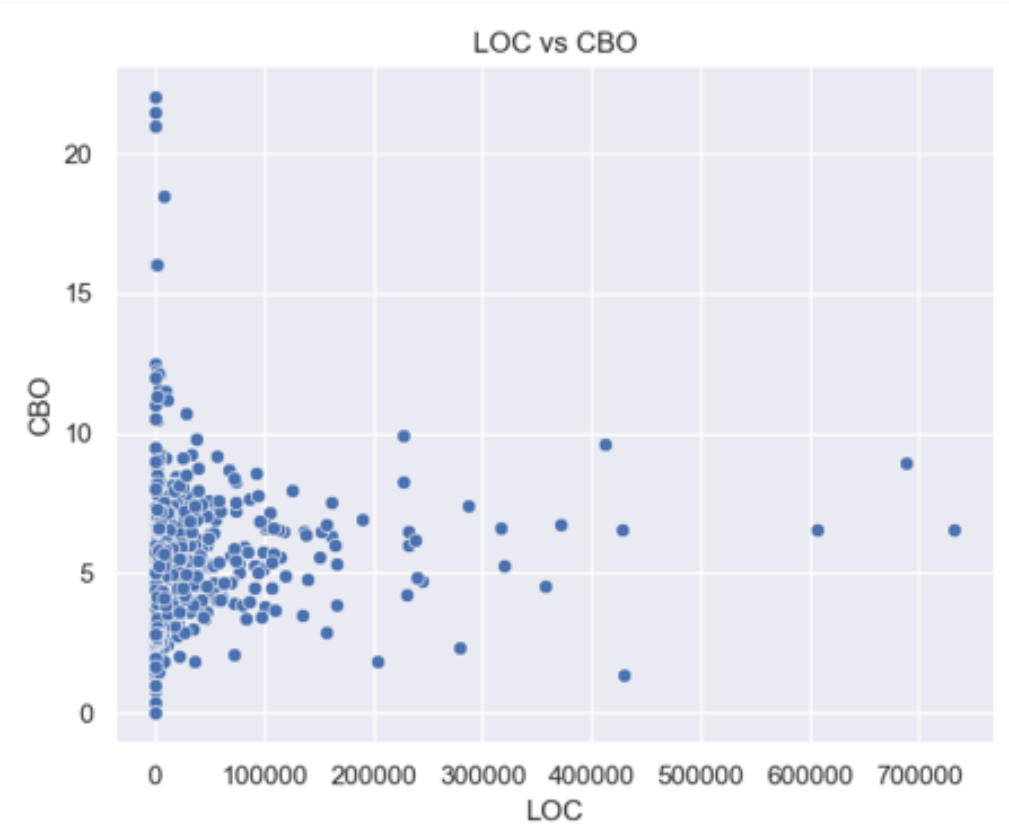


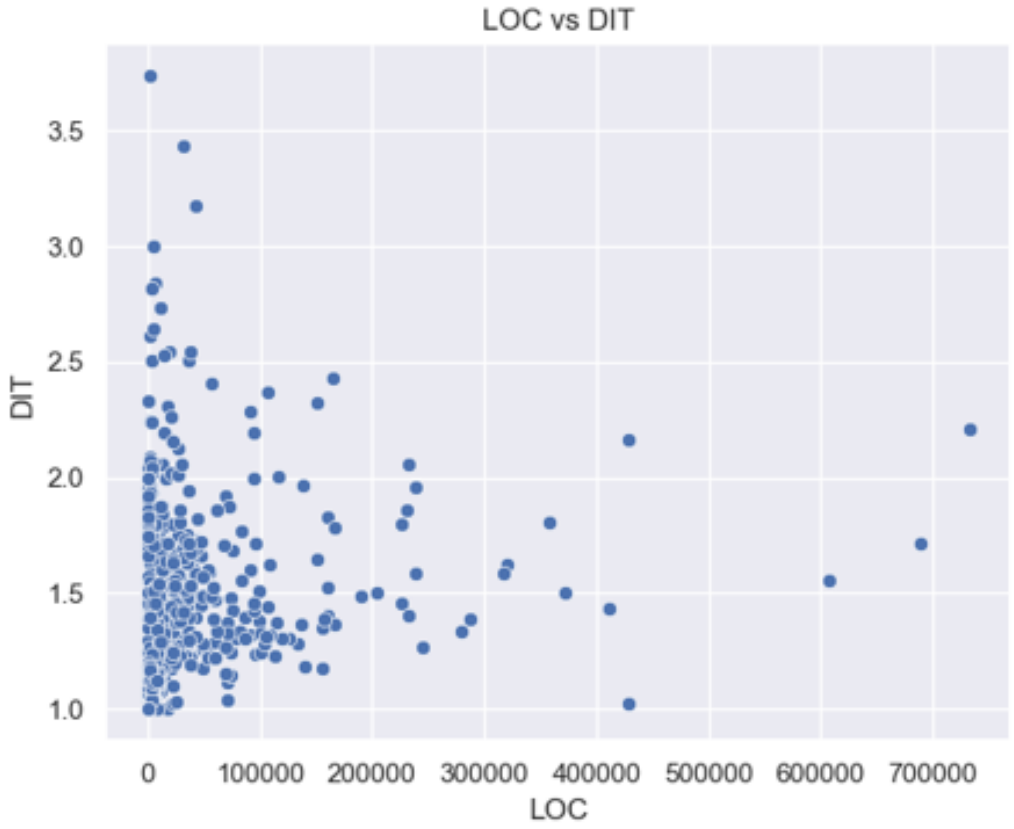


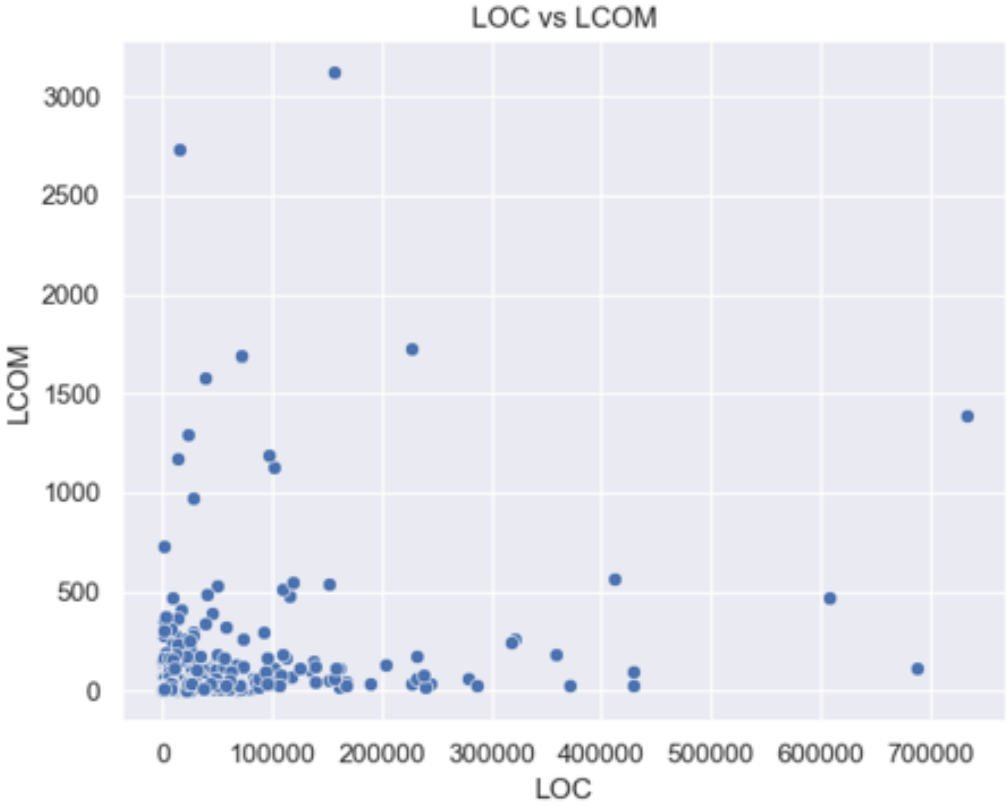


RQ04 – Tamanho x Qualidade (LOC vs Qualidade)

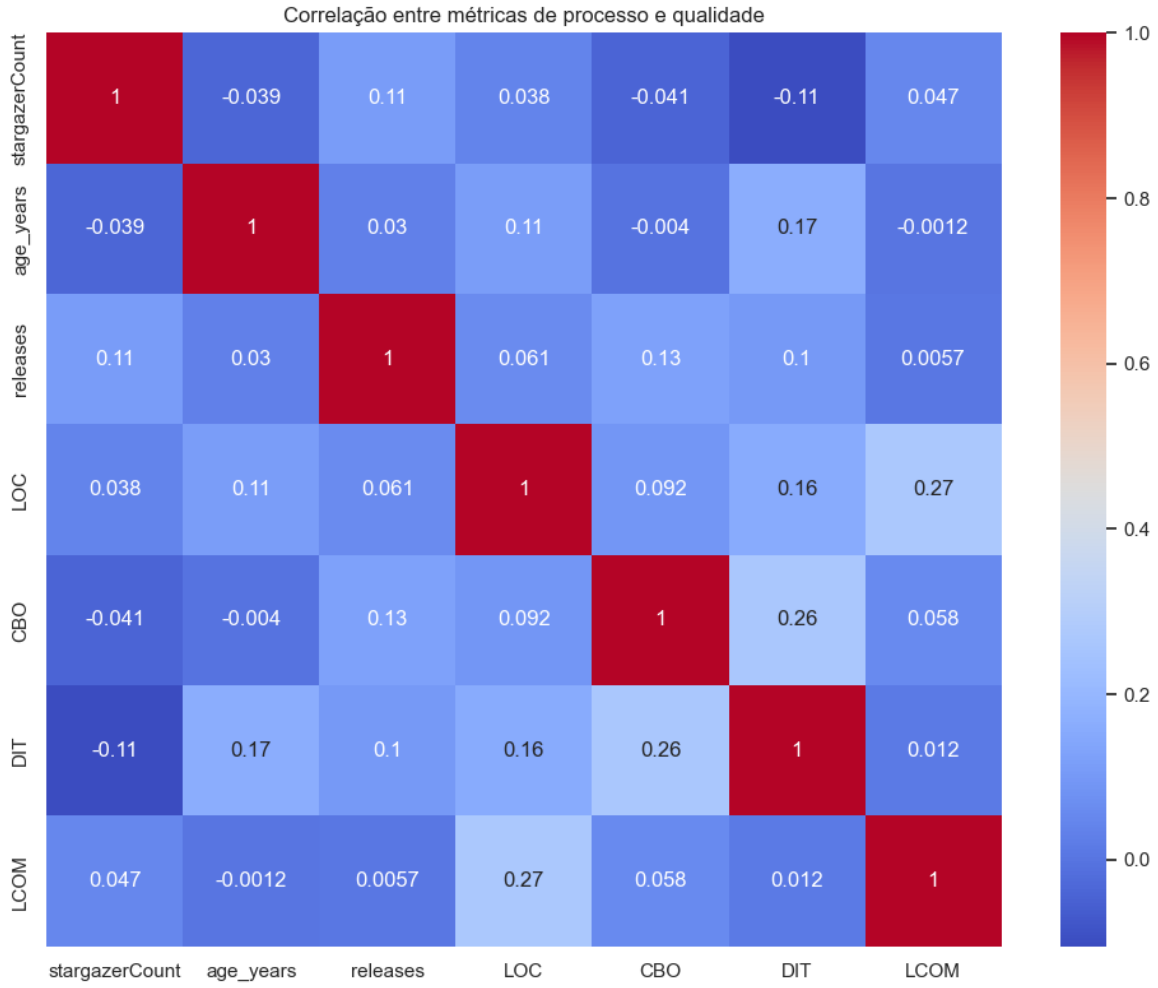
O gráfico abaixo revela uma tendência de crescimento clara: quanto maior o LOC, maior o LCOM (perda de coesão).




LOC vs DIT
DIT
3.5
3.0
2.5
2.0
1.5
1.0
0
100000
200000
300000
400000
500000
600000
700000
LOC


LOC vs LCOM
LCOM
3000
2500
2000
1500
1000
500
0
0
100000
200000
300000
400000
500000
600000
700000
LOC

A imagem abaixo confirma que a correlação mais forte é entre métricas de volume (LOC) e métricas de complexidade/acoplamento.



Discussão e interpretação

Explicação

Sistemas maiores tendem a acumular "classes divinas" (God Classes), o que explica o aumento do LCOM. A popularidade é um fenômeno social, não técnico.

Limitações

A análise não diferencia bibliotecas (libs) de aplicações finais, que possuem padrões de acoplamento distintos.

Trade-offs

Projetos com muitas releases (alta atividade) ganham funcionalidade, mas pagam o preço em acoplamento (CBO) crescente.

Decisão e recomendações

Decisão

Iterar. Recomenda-se que grandes projetos adotem políticas estritas de quebra de classes após certo limiar de LOC.

Monitoramento pós-lançamento

Integrar a métrica LCOM ao Dashboard de CI dos projetos analisados.

Custos e impactos

Custo computacional

Aproximadamente 6 horas de processamento para clonar e analisar 1.000 repositórios.

Risco técnico e operacional

Dependência da disponibilidade da API do GitHub e integridade dos códigos-fonte (alguns projetos podem não compilar).

Ética, privacidade e conformidade

Os dados são públicos sob licenças open-source. Nenhuma informação pessoal de desenvolvedores foi processada além dos metadados públicos dos repositórios.

Reprodutibilidade

Para que seja possível reproduzir o experimento, basta clonar nosso repositório do Github, criar um token do próprio Githubs e executar o main.py.